Online Processing of Influence Maximization Revisited: Killing Two Birds with One Stone (Technical Report)

Anonymous Author(s)

ABSTRACT

Influence maximization (*IM*) is a classical and intensively studied problem in the graph data mining with a lot of applications in vital markets and social advertising. *Online processing of influence maximization* (*OPIM*) has drawn more and more attention in the recent years and has become the de facto paradigm for IM processing. It allows the user to ask for the quality that it can achieve so far at any time. In this sense, OPIM is very similar to the online aggregation in the database query and enjoys better interactiveness and flexibility and the nature of on-the-fly quality estimation also help OPIM algorithms terminate earlier once the desired accuracy is achieved and as such, OPIM achieves the state-of-the-art performances for IM. Two crucial and inevitable components in OPIM are (1) online finding of a size-k seed set S^* of nodes with the high influence and (2) the online estimation of the approximation ratio of the currently obtained set of nodes. We observe that all existing OPIM algorithms with favorable accuracy guarantee achieve the two objectives separately by sampling two independent sets of samples called *RR sets*. Besides, they also suffers from their static sampling schedule in which the sample size are simply doubled in each iteration.

Motivated by this, we propose an efficient algorithm, namely *Online Processing Influence Maximization with Rademacher Average* (*OPERA*), for OPIM. Its key feature is "killing two birds with one stone" which adopts the same set of samples for both node set finding and the approximation ratio estimation. As such, each sample in our algorithm contributes for both the node set finding and the approximation ratio estimation and thus, the sampling efficiency is highly boosted. To this end, we incorporate a fundamental concept called *Rademacher Average* from statistical learning theory in our quality estimation. We also proposes a corresponding efficient algorithm for estimating the quality by using convex optimization. Based on our estimation, we propose an error-aware dynamic sampling schedule which helps us further reduce the total sample size. Our experiments show that our algorithm significantly outperforms the state-of-the-art algorithms (including both existing online and offline algorithms) in terms of total sample size and running time by orders of magnitudes.

29th ACM SIGKDD, Aug. 03–05, 2023, Long Beach, CA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

KEYWORDS

Social Networks, Influence Maximization, Rademacher Average, Random Sampling

ACM Reference Format:
Anonymous Author(s). 2023. Online Processing of Influence Maximization Revisited: Killing Two Birds with One Stone (Technical Report). In *Proceedings of 29th ACM SIGKDD.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Influence maximization (*IM*) is a classical problem and has received a lot of attention from both academia and industry. Given a network $G(V, E)$ and a positive integer k, it asks for a size-k node set which has the largest number of influenced users under a cascade model C. In IM, the cascade model is adopted to capture the well known *Word-of-Mouth* effect in viral marketing and social advertising which reveals that the users adopt certain products and receives opinions due to the influence of their friends on the social networks. To make a viral marketing campaign/advertising successful, it is crucial to select a set of influential users which serves as the seeds to spread/propagate the information [14, 30]. Due to the limited budget, the number of the selected users must be upper bounded by a positive integer k.

The seminal work [24] of IM proved that the problem is NP-hard. They studied two cascade models, namely *Independent-Cascade* (*IC*) and *Linear-Threshold* (*LT*), also gave the first attempt to tackle IM which provides $O(1 - \frac{1}{e} - \epsilon)$-approximate solution for both models, where ϵ is a user-specified error parameter. Then, a plethora of subsequent research efforts [1, 7–9, 11, 12, 12, 13, 15–19, 22, 23, 25, 28, 29, 33, 36, 38, 40, 41, 46, 47] were devoted to improve the efficiency of processing offline.

Later on, offline processing of IM was found to be lack of interactiveness and flexibility [34] since users can not get any feedback or indicator during their process (which may take a significantly long time when the quality requirement is stringent) and have no choice but wait until the algorithm terminates which leads to poor user experience. Besides, the lack of online monitoring the currently achieved quality also renders offline IM suffer from their overestimated sample size due to their worst-case analysis. Thus, *online processing of IM* (*OPIM*) draws more and more attention in recent years [3, 20, 21, 34]. OPIM allows the users to inquire about the quality that it can achieve so far and as such, the users have the choice to terminate the algorithm earlier when the quality is good enough and obtain the corresponding seed set. OPIM enjoys better interactiveness and flexibility and besides, the nature of online quality estimation helps OPIM algorithms terminates earlier than offline algorithms for the same quality assurances. As

such, OPIM algorithms even have the state-of-the-art performances in the offline setting. [3] provides the first attempt of solving OPIM but the approximation ratio that can be achieved by their algorithm is at most $\frac{1}{4}$ which is too small for real applications. Later on, [20, 21, 34] provides $(1-\frac{1}{e}-\epsilon)$-approximate algorithms with high probability (w.h.p.), in which the gap ϵ is the user-specified parameter regarding the desired quality. OPIM has two important and inevitable building blocks. One is the component of finding the size-k seed set S^* and the other one is online estimation of the approximation ratio of the currently obtained seed set. Both components are achieved by using a technique called *Reverse Influence Sampling*, in which a set of *reverse reachable sets* (*RR sets*) is sampled. Each RR set is a randomly generated sub-graph of G. We observe that all existing OPIM algorithms with favorable accuracy guarantee [20, 21, 34] samples two sets of RR sets independently in each iteration. The first set of RR sets, denoted by $\mathcal{R}_1$ is utilized to find size-k seed set S^*. To derive the approximation ratio of S^*, it is necessary to compute the upper bound of the influence of S^{OPT} (i.e, the optimal solution) and also the lower bound of the influence of S^*. As pointed out in [20, 34], the lower bound of the influence of S^* can not be derived from $\mathcal{R}_1$ due to the limitations of the concentration bounds (which their analysis relies on). As such, another set $\mathcal{R}_2$ of RR sets with the same size must be sampled independently for the estimation (although $\mathcal{R}_2$ does not contribute to the seed set selection at all) which leads to low sampling efficiency. Besides, they also suffer from their static sampling schedule in which the sizes of $\mathcal{R}_1$ and $\mathcal{R}_2$ are simply doubled in each iteration. This sampling schedule wastes a large number of samples when the empirical approximation ratio is very close to the desired one.

Motivated by this, we propose an OPIM algorithm, namely *Online Processing Influence Maximization with Rademacher Average* (*OPERA*), and its key feature is "killing two birds with one stone". Specifically, OPERA selects the seed set and estimates its approximation ratio by using the same set $\mathbb{R}$ of RR sets (i.e., each RR set sampled contribute to both seed set selection and the quality estimation) which help us achieve a high sampling efficiency. To this end, we incorporate a fundamental concept called *Rademacher Average* from the statistical learning theory which was originally proposed to analysis the convergence of a family of functions. The Rademacher average estimates the maximum deviation of the estimated influence of any size-k seed set from its real value by using a set of randomly generated variables. To make the computation of the Rademacher average more efficiently, we also propose a convex optimization algorithm for the computation of the upper bound of this Rademacher average. Besides, we propose a dynamic quality-aware sampling schedule which carefully increase the sample size when the quality approaches the desired one and help us save the number of samples. Our algorithm also provides a $(1-\frac{1}{e}-\epsilon)$-approximate solution w.h.p.

Our contributions are summarized as follows. Firstly, we propose an Radamacher average-based method for online estimating the quality of our selected seed set. We also develop a convex optimization algorithm for efficiently computing the required Rademacher average in this estimation algorithm. Secondly, based on our quality estimation algorithm, we propose an OPIM algorithm *OPERA* which select the seed set and estimates its quality by using the same set of RR sets. In our algorithm, we adopt a dynamic quality-aware sampling schedule which significantly reduces the number of total samples. Our algorithm has the same approximation guarentee as the state-of-the-art algorithms which finds a $(1-\frac{1}{e}-\epsilon)$-approximate solution w.h.p. Thirdly, we conduct intensive empirical study and our experimental results show that our algorithm significantly outperforms the existing IM algorithms by orders of magnitudes in terms of both the sample size and the running time with the same quality guarantee. The source code will be made publicly available after the publication of the paper.

The remainder of the paper is organized as follows. Section 2 provides the problem statement and Section 3 presents the existing studies. Section 4 presents our proposed algorithm and Section 5 demonstrates our empirical study. Finally, Section 6 concludes the paper.

2 PROBLEM DEFINITION

Let $G(V, E)$ denote a social network, where V is the set of vertices and E is the set of edges. Each vertex represents a user and each edge represents the connection between two users. For each edge (u, v), there is an associated probability $p(u, v) \in [0, 1]$ called *propagation probability*. Given a set S of vertices in G, the influence $\mathbb{I}(S)$ is calculated as follows by using a discrete-time stochastic process called *Cascade Model*.

- In timestamp 0, each vertex in S is *activated* and any vertex outside S is *inactivated*.
- In timestamp i, where i is at least 0, if a vertex is *activated*, it has a chance to *activate* its out-neighbors based on the *Cascade Model* (to be shown later soon) in timestamp $i+1$. After timestamp $i+1$, it cannot activate other vertices anymore in later timestamps.
- Each *activated* vertex keeps activated.
- The propagation terminates when each activated vertex can not activate any inactivated vertex.

The IM problem adopts two Cascade models, namely *Independence Cascade* (*IC*) *model* and *Linear Threshold* (*LT*) *model* which are detailed as follows [24].

IC Model: Suppose that a node u is activated in timestamp i. Then, in timestamp $i+1$, each inactivated neighbor v of u will be activated with the probability $p(u, v)$.

LT Model: Suppose that for each node v, there exists a predefined threshold λ_v in the range of $[0, 1]$ and the sum of the propagation probabilities of its incoming edges is no more than 1. If v is inactivated in timestamp i, it will be activated in timestamp $i+1$ if $\sum_{u \in A} p(u, v) \geq \lambda_v$, where A denotes the set of in-neighbors of u which are activated in timestamp i.

We denote the number of activated vertices by $I_C(S)$ in the above stochastic process, where C is one propagation instance of the stochastic process. We denote the Cascade Model (i.e., the stochastic process) by $\mathbb{C}$. Then, the *expected influence* $\mathbb{I}_{\mathbb{C}}(S)$

of the given set S under the model $\mathbb{C}$ is $\mathbb{E}_{C\in\mathbb{C}}I_C(S)$. Given a network G and a positive integer k, *Influence Maximization* (*IM*) aims to find a size-k seed set S^* which has the maximum expected influence.

Online Processing of Influence Maximization (*OPIM*) further improves the flexibility and interactiveness of IM and allows the user to enquiry about the currently achieved accuracy and seed set at any time during the processing of IM. We provide the formal definition of this problem as follows.

Problem 1 (Online Processing of Influence Maximization (OPIM)). *Given a network G and a positive integer k and a streaming sequence of timestamps t_1, t_2, t_3, ..., OPIM asks for a seed set S_i at timestamp t_i and an approximation guarantee α_i, such that S_i achieves α_i-approximation. The objective is to maximize α_1, α_2, α_3, ...*

3 RELATED WORK

3.1 Reverse Influence Sampling (RIS)

Most existing efficient IM methods are based on a sampling technique called Reverse Influence Sampling (RIS), which was proposed by Borgs et al. [3]. The core idea of this technique is the concept of random reverse reachable (RR) set. A random RR set R is constructed in two steps: (i) uniformly sample a node $v \in V$; (ii) reversely sample the set R of nodes that can activate v, such that for each node $u \in V$, the probability that it is included in R equals the probability that u can activate v. This set R is denoted as a reverse reachable set of v. Then, it proposes the following theorem which connects the random RR set and the influence of a seed set S.

Theorem 1. *Given a set S of vertices and a random reverse reachable set R_r generated under a Cascade model $\mathbb{C}$, then we have*

$$\mathbb{I}_{\mathbb{C}}(S) = n \cdot Pr[S \cap R_r \neq \emptyset]$$

3.2 Offline Influence Maximization Algorithms

Kempe et al. [24] propose the first seminal work of IM and it proved that IM is an NP-hard problem. Let n and m denotes the number of nodes and edges in the network. It also develop a $(1-\frac{1}{e}-\epsilon)$-approximate algorithm with the time complexity $\Omega(k\cdot m\cdot n\cdot poly(\frac{1}{\epsilon}))$ which is prohibitively expensive for sizable networks.

Borgs. et al. [3] made a theoretical breakthrough with the aforementioned RIS technique which reduces the time complexity to $O(k\cdot(m+n)\frac{log^2 n}{\epsilon^3})$ while still providing $(1-\frac{1}{e}-\epsilon)$-approximate solution. The algorithm involves two phases and in the first phase, it samples a set $\mathcal{R}$ of random RR sets. Given a set S of vertices and an RR set R, we call that R is *covered by* S if S has overlap with R. We use $C_{\mathcal{R}}(S)$ to denote the number of RR sets in $\mathcal{R}$ covered by S. According to Theorem 1, $\frac{n}{|\mathcal{R}|}\cdot C_{\mathcal{R}}(S)$ is an unbiased estimation of $\mathbb{I}_{\mathbb{C}}(S)$. Thus, in the second phase, it performs a greedy algorithm (shown as Algorithm 1) to select a size-k seed set S^* which covers the maximal numbers of RR sets. According to [3], S^* is a $(1-\frac{1}{e}-\epsilon)$-approximate solution. Then, a plethora of subsequent works [1, 7–9, 11–13, 15–19, 22, 23, 25, 28, 29, 33, 36, 38, 40, 41, 46, 47] improved the efficiency of the offline IM processing. Most of them are heuristic-based and failed to provide any approximation guarantee and among them, [12, 22, 28, 40, 41] focused on reducing the total number of RR sets required with the same approximation guarantee.

Algorithm 1 GreedyIM(G, $\mathcal{R}$, k)

Input: Graph $G=(V,E)$, a set $\mathcal{R}$ of RR sets and an integer k
Output: A set S^* of k nodes
1: Initialize S^* to be $\emptyset$;
2: **for** i from 1 to k **do**
3: $\quad u \leftarrow \arg\max_{v\in V\setminus S^*}(C_{\mathcal{R}}(S^*\cup\{u\}) - C_{\mathcal{R}}(S^*\cup\{u\}))$
4: $\quad S^* \leftarrow S^*\cup\{u\}$
5: **end for**

All algorithms in this category only samples RR sets in one lump and does not provide any data-dependent accuracy estimation method by which the approximation ratio of the currently obtained seed set can be computed. As such, these algorithms always over-estimate the number of RR sets needed and fail to provide the flexibility and interactiveness.

3.3 Online Influence Maximization Algorithms

It was observed in [3, 20, 21, 34] that the offline IM lack of interactiveness and do not respond until they finish the whole processing which may largely degrade the user experience, esp. in the cases where their running time is significantly large. Motivated by this, *Online Processing of IM* (*OPIM*) was proposed [3, 20, 21, 34] where the user can inquiry about the accuracy that it can achieve so far. The user is allowed to terminate the algorithm at any time when the quality currently achieved is satisfiable and at the time when the algorithm is terminated, the corresponding seed set is returned. Thus, OPIM algorithms [20, 21, 34] achieved the state-of-the-art performances among all IM algorithms due to their online processing nature.

Borgs. et al. [3] proposed an online version of the RIS-based algorithm which is the first OPIM algorithm. This algorithm keeps generating RR sets and performs the greedy algorithm to find a seed set when the number γ of edges examined during the RR set generation is equal to the power of 2. For each seed set selected, the approximation ratio is calculated as $\min\{\frac{1}{4},\beta\}$, where β is defined as follows.

$$\beta = \frac{\gamma}{1492992\cdot(n+m)\log n}$$

From the above equation, we notice that the approximation ratio is at most $\frac{1}{4}$ which is too small for real applications. Besides, as discussed in [34], the approximation ratio above is too loose to be adopted in practice. For example, to achieve 0.1-approximate solution on a graph with only 10^5 nodes and 10^6 edges, it requires $2\cdot 10^{12}$ edges to be examined which incur prohibitively large overhead.

Later on, Tang et al. [34] proposed an iterative algorithm called *OPIM-C*. In each iteration, it first samples a set $\mathcal{R}_1$ of RR sets, based on which, it invoked the Greedy algorithm

to obtain the size-k seed set S^* and also estimate the upper bound of $\mathbb{I}_{\mathbb{C}}(S^{OPT})$. Since it is not possible to estimate the lower bound of $\mathbb{I}_{\mathbb{C}}(S^*)$ by $\mathcal{R}_1$ based on the traditional concentration inequalities, it samples a second set $\mathcal{R}_2$ (with the same size as $\mathcal{R}_1$) of RR sets for estimating the lower bound of $\mathbb{I}_{\mathbb{C}}(S^{OPT})$. As such, the lower bound of the approximation ratio of S^* can be derived by using the two bounds. For a given error parameter ϵ, both OPIM-C and SUBSIM can return $(1-\frac{1}{e}-\epsilon)$-approximate solution w.h.p. However, in OPIM-C, $\mathcal{R}_2$ does not contribute to the selection of S^* at all which renders it suffer from the low sampling efficiency. Later on, SUBSIM [20, 21] further improved the random RR sets generation algorithm involved in OPIM-C but keeps other operations intact (i.e., the low sampling efficiency problem still exists in SUBSIM). Besides, OPIM-C and SUBSIM also suffers from their static sampling schedule which is unaware of the currently achieved quality. In particular, they simple double the sample size in each iteration which incurs the waste of samples, esp. when the currently achieved quality is very close to the user desired one where it suffices to only increase the sample size a bit.

3.4 Other Related Studies

Many recent research works [2, 6, 10, 26, 27, 35, 37, 39, 42, 44] also investigated some variants of the IM problem which consider other factors such as topics of interest, user-input keywords, geo-location, etc. or studied the settings of the propagation probability. But they have different problem settings and their techniques can not apply for our problem.

4 OUR PROPOSED ALGORITHM

Algorithm 2 demonstrates the framework of our proposed algorithm *OPERA*. In Line 1 and Line 2, we initialize the seed set S^* to be returned, the set $\mathcal{R}$ of RR sets and the initial sample size. We will discuss in detail in Section 4.3 on the setting of the initial sample size. From Line 3 to Line 12, we have an iterative procedure which progressively sample RR sets and select a seed set correspondingly until the desired approximation ratio is achieved. In Line 4 and Line 5, we randomly sample a set $\Delta\mathcal{R}$ of RR sets by using the standard technique [20, 21] and inserts the RR sets sampled into $\mathcal{R}$. Based on $\mathcal{R}$, we use the greedy algorithm (i.e., Algorithm 1) to obtain a size-k seed set S^* in Line 5. Line 7 finds the approximation ratio θ of S^* and we will present in Section 4.2 our proposed approximation ratio estimation algorithm which is inspired by a key concept called *Rademacher Average* from the statistical machine learning. In Line 8 to Line 10, we safely terminate and return S^* if θ is at least our desired approximation ratio. In Line 11, we calculate the sample size Δl for the next iteration which we will discuss in detail in Section 4.3.

This section is organized as follows. In Section 4.1, we introduce some important notations and concepts. Section 4.2 and Section 4.3 present our approximation ratio estimation algorithm and our sample schedule (i.e., sample size update algorithm) respectively.

4.1 Notations and Concepts

We denote the set of all possible random reverse reachable sets (RR sets) by $\mathcal{D}$ and denote the set containing all possible size-k seed sets by $\mathcal{S}$.

Consider a seed set $S \in \mathcal{S}$. Let $f_S(R)$ denote a variable whose value is in $\{0, 1\}$, where R is a random reverse reachable set and S is a size-k seed set. It is equal to 1 if the given sample R is a positive sample w.r.t. S (i.e., R has intersection with S) and otherwise, it is equal to 0. Thus, we obtain that

$$Pr[S \cap R_r \neq \emptyset] = \frac{1}{|\mathcal{D}|} \sum_{R \in \mathcal{D}} f_S(R) \tag{1}$$

Algorithm 2 OPERA(G, ϵ, k, η)

Input: Graph $G = (V, E)$, an error parameter ϵ, an integer k and a failure probability threshold η
Output: A size-k seed set S^* with approximation ratio more than $1-\frac{1}{e}-\epsilon$ with probability at least $1-\eta$

1: $S^* \leftarrow \emptyset, \mathcal{R} \leftarrow \emptyset$;
2: $\Delta l \leftarrow \ln \frac{2}{\eta} / (2 \cdot \frac{1}{\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e})}), l \leftarrow \Delta l$;
3: **for** i from 1 to ∞ **do**
4: $\quad \Delta\mathcal{R} \leftarrow$ Randomly sample Δl RR sets;
5: $\quad \mathcal{R} \leftarrow \mathcal{R} \cup \Delta\mathcal{R}, l \leftarrow |\mathcal{R}|$;
6: $\quad S^* \leftarrow \text{greedyIM}(G, \mathcal{R}, k)$;
7: $\quad \theta, \omega^* \leftarrow \text{getApproRatio}(\mathcal{R}, S^*, \eta, l)$;
8: $\quad$ **if** $\theta \geq (1-\frac{1}{e}-\epsilon)$ **then**
9: $\quad\quad$ return S^*;
10: $\quad$ **end if**
11: $\quad \Delta l \leftarrow \text{getSampleSize}(\mathcal{R}, S^*, \omega^*, \epsilon, \eta, l)$;
12: **end for**

Given a set $\mathcal{R}$ of RR sets and a seed set S, the estimated probability $\tilde{Pr}[S \cap R_r \neq \emptyset]$ is calculated as follows.

$$\tilde{Pr}[S \cap R_r \neq \emptyset] = \frac{1}{|\mathcal{R}|} \sum_{R \in \mathcal{R}} f_S(R) \tag{2}$$

The maximum error of the estimated probability of any size-k seed set S is shown in the following equation.

$$\sup_{S \in \mathcal{S}} |\tilde{Pr}[S \cap R_r \neq \emptyset] - Pr[S \cap R_r \neq \emptyset]| \tag{3}$$

4.2 Online Estimation of approximation ratio

This section demonstrates our approximation ratio estimation algorithm and we first present a key concept called *Rademacher Average*. Rademacher Average is a very core concept in the statistical machine learning theory [31, 43] which was proposed to study the convergence rate of a set of sample averages to their expectations. We present here only the definitions and results that we use in our work and we refer the readers to the book by Shalev-Shwartz and Ben-David [32] for the in-depth presentations and discussions. Consider l independent random variables $\sigma_1, \sigma_2, ..., \sigma_l$, where $l = |\mathcal{R}|$. Each random variable σ_i ($i \in [1, l]$) takes value 1 with probability 0.5 and takes the value -1 with probability 0.5. Let σ denote the vector

containing all the random variables (i.e., $\sigma = (\sigma_1, \sigma_2, ..., \sigma_i, ..., \sigma_l)$) The Rademacher average of a set $\mathcal{S}$ of all possible size-k seed sets and a set $\mathcal{R} = \{\mathcal{R}_1, \mathcal{R}_2, ..., \mathcal{R}_i, ..., \mathcal{R}_l\}$ of RR sets sampled is defined as follows.

$$\mathbb{R}(\mathcal{S}, \mathcal{R}) = \mathbb{E}_\sigma[\sup_{S \in \mathcal{S}}(\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i))] \tag{4}$$

The following theorem connects the maximum error shown in Equation (3) that we would like to derive and the Rademacher Average.

Theorem 2. *Let $\eta \in (0, 1)$ be a given failure probability and let $\mathcal{R}$ be a set of uniformly and independently sampled random RR sets. R_r denote an RR set generated under the Cascade Model C. With the probability at least $1 - \eta$, we have*

$$\sup_{S \in \mathcal{S}} |\tilde{Pr}[S \cap R_r \neq \emptyset] - Pr[S \cap R_r \neq \emptyset]| \leq 2\mathbb{R}(\mathcal{S}, \mathcal{R}) + \frac{\ln \frac{2}{\eta}}{2l} \tag{5}$$

Proof. We first recite McDiarmid's Inequality (Bennett form) as follows.

Theorem 3 (McDiarmid's Inequality (Bennett form) [45]). *Consider a function $f : X^l \to \boldsymbol{R}$, where $\boldsymbol{R}$ denotes the real domain and $|f(x_1, x_2, ..., x_i, ..., x_l) - f(x_1, x_2, ..., x_i', ..., x_l)| \leq c_i$ for each i in $[1, l]$ and c_i is a positive real-valued number. Consider a set x of independent random samples from X, denoted by $x_1, x_2, ..., x_l$. Then, we have*

$$\begin{aligned} &Pr[f(x_1, x_2, ..., x_i, ..., x_l) - \mathbb{E}[f(x_1, x_2, ..., x_i, ..., x_l)] \geq \epsilon] \\ \leq &exp(-\frac{\epsilon}{2B} \cdot \log(1 + \frac{\epsilon \cdot B}{\tilde{\sigma}^2})) \end{aligned}$$

In the above inequality, the variables B, V_i and $\tilde{\sigma}^2$ are defined as follows.

$$\begin{aligned} B &= \max_i \sup_{x \setminus x_i} |f(x_1, x_2, ..., x_i, ..., x_l) - \mathbb{E}_i[f(x_1, x_2, ..., x_i, ..., x_l)| \\ V_i &= \sup_{x \setminus x_i} (f(x_1, x_2, ..., x_i, ..., x_l) - \mathbb{E}_i[f(x_1, x_2, ..., x_i, ..., x_l))^2 \\ \tilde{\sigma}^2 &= \sum_i V_i \end{aligned}$$

We use $m_{\mathcal{R}}(S)$ and $m_{\mathcal{D}}(S)$ to denote $\tilde{Pr}[S \cap R_r \neq \emptyset]$ and $Pr[S \cap R_r \neq \emptyset]$ for better conciseness. In our problem, we set the function f to be $\sup_S |\tilde{Pr}[S \cap R_r \neq \emptyset] - Pr[S \cap R_r \neq \emptyset]| = \max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)|$ and correspondingly, we have that $\mathbb{E}_S[f] = \mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)|]$. Since each sample in S only contribute at most $\frac{1}{l}$ in f, we obtain that $B = \frac{1}{l}$ and $V_i = 1$. Thus, we obtain that $\tilde{\sigma}^2 = l$. We proceed to obtain that

$$\begin{aligned} &Pr[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)| - \mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)| \geq \epsilon] \\ \leq &exp(-\frac{\epsilon}{2B} \cdot \log(1 + \frac{\epsilon \cdot B}{\tilde{\sigma}^2})) \leq exp(-\frac{\epsilon \cdot l}{2} \cdot \log(1 + \frac{\epsilon}{l^2})) \\ \leq &exp(-\frac{\epsilon \cdot l}{2}) \text{ (since } \tfrac{\epsilon}{l^2} \leq 1) \end{aligned}$$

Thus, we obtain that with the probability $1 - \eta$, we have

$$\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)| - \mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)|] \leq \frac{\ln \frac{2}{\eta}}{2l} \tag{6}$$

Next, we focus on the calculation of $\mathbb{E}_S[f] = \mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)|]$. Consider anther set of samples $\mathcal{R}'$ from $\mathcal{D}$, denoted by $\{R_1', R_2', ..., R_l'\}$. We further obtain that

$$\begin{aligned} &\mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)|] = \mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - E_{\mathcal{R}'} m_{\mathcal{R}'}(S)|] \\ \leq &\mathbb{E}_{\mathcal{R}, \mathcal{R}'}[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{R}'}(S)|] \text{(By Jensen's Inequality)} \\ \leq &\mathbb{E}_{\mathcal{R}, \mathcal{R}'}[\max_S |\frac{1}{l}\sum_{R \in \mathcal{R}} f_S(R) - \frac{1}{l}\sum_{R \in \mathcal{R}'} f_S(R)|] \\ \leq &\mathbb{E}_{\mathcal{R}, \mathcal{R}', \sigma}[\max_S |\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot (f_S(\mathcal{R}_i) - f_S(\mathcal{R}_i'))|] \\ \leq &\mathbb{E}_{\mathcal{R}, \sigma}[\max_S |\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i)|] + \mathbb{E}_{\mathcal{R}', \sigma}[\max_S |\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i')|] \\ \leq &2 \cdot \mathbb{R}(\mathcal{S}, \mathcal{R}) \end{aligned} \tag{7}$$

By Inequality (6) and Inequality (7), we obtain that with the probability $1 - \eta$, we have

$$\begin{aligned} &\sup_S |\tilde{Pr}[S \cap R_r \neq \emptyset] - Pr[S \cap R_r \neq \emptyset]| = \max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)| \\ \leq &\mathbb{E}_S[\max_S |m_{\mathcal{R}}(S) - m_{\mathcal{D}}(S)|] + \frac{\ln \frac{2}{\eta}}{2l} \leq 2 \cdot \mathbb{R}(\mathcal{S}, \mathcal{R}) + \frac{\ln \frac{2}{\eta}}{2l} \end{aligned}$$

□

Algorithm 3 getApproRatio($\mathcal{R}, S^*, \eta, l$)

Input: A set $\mathcal{R}$ of RR sets, a size-k seed set S^*, a failure probability threshold η, the total sample size l

Output: The approximation ratio of S^*

1: $\tilde{Pr}[S^* \cap R_r \neq \emptyset] \leftarrow \frac{1}{|\mathcal{R}|}\sum_{R \in \mathcal{R}} f_S(R)$;
2: $w(x) \leftarrow \frac{1}{x} \cdot \ln(|\mathcal{V}(\mathcal{R})| \cdot \exp(\frac{x^2 \cdot \tilde{Pr}[S^* \cap R_r \neq \emptyset]}{2 \cdot l}))$;
3: $\omega^* \leftarrow \min_{x \in \mathbf{R}^+} w(x)$;
4: $\mathcal{E}^+ \leftarrow 2 \cdot \omega^* + \frac{\ln \frac{2}{\eta}}{2l}$;
5: Return $\frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}^+}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1 - \frac{1}{e}) + \mathcal{E}^+}, \omega^*$;

Let $\mathcal{E}$ denote $2 \cdot \mathbb{R}(\mathcal{S}, \mathcal{R}) + \frac{\ln \frac{2}{\eta}}{2l}$ (which is the right hand side of Inequality (5)). Let S^* be the seed set returned by our algorithm and let S^{OPT} denote the optimal size-k seed set. We further obtain that

Theorem 4. *Let $\eta \in (0, 1)$ be a given failure probability and let $\mathcal{R}$ be a set of uniformly and independently sampled RR sets. With the probability at least $1 - \eta$, we have the approximation ratio*

$$\theta = \frac{\mathbb{I}_{\mathbb{C}}(S^*)}{\mathbb{I}_{\mathbb{C}}(S^{OPT})} \geq \frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1-\frac{1}{e}) + \mathcal{E}} \tag{8}$$

Proof. By Theorem 2, we obtain that with the probability at least $1-\eta$, we have

$$\begin{aligned} |\tilde{Pr}[S^* \cap R_r \neq \emptyset] - Pr[S^* \cap R_r \neq \emptyset]| \leq \mathcal{E} \\ |\tilde{Pr}[S^{OPT} \cap R_r \neq \emptyset] - Pr[S^{OPT} \cap R_r \neq \emptyset]| \leq \mathcal{E} \end{aligned} \tag{9}$$

Thus, we obtain that

$$\begin{aligned} Pr[S^* \cap R_r \neq \emptyset] \geq \tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E} \\ Pr[S^{OPT} \cap R_r \neq \emptyset] \leq \tilde{Pr}[S^{OPT} \cap R_r \neq \emptyset] + \mathcal{E} \end{aligned} \tag{10}$$

Let S' denote the size-k seed set which covers the largest number of RR sets in $\mathcal{R}$. By the greedy algorithm [3], we obtain that

$$\begin{aligned} \tilde{Pr}[S^* \cap R_r \neq \emptyset] \geq & (1-\frac{1}{e})\tilde{Pr}[S' \cap R_r \neq \emptyset] \\ \geq & (1-\frac{1}{e})\tilde{Pr}[S^{OPT} \cap R_r \neq \emptyset] \end{aligned} \tag{11}$$

By Inequality (10) and Inequality (11), we obtain that the approximation ratio

$$\begin{aligned} & \frac{\mathbb{I}_{\mathbb{C}}(S*)}{\mathbb{I}_{\mathbb{C}}(S^{OPT})} = \frac{n \cdot Pr[S^* \cap R_r \neq \emptyset]}{n \cdot Pr[S^{OPT} \cap R_r \neq \emptyset]} = \frac{Pr[S^* \cap R_r \neq \emptyset]}{Pr[S^{OPT} \cap R_r \neq \emptyset]} \\ \geq & \frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}}{\tilde{Pr}[S^{OPT} \cap R_r \neq \emptyset] + \mathcal{E}} \geq \frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1-\frac{1}{e}) + \mathcal{E}} \end{aligned}$$

□

As Inequality (5) reveals, the key component in $\mathcal{E}$ is the Rademacher average $\mathbb{R}(\mathcal{S}, \mathcal{R})$. But it is computationally prohibitive to compute $\mathbb{R}(\mathcal{S}, \mathcal{R})$ directly with its definition due to the exponential time complexity of enumerating all possible values of σ_i. Alternatively, we derive an upper bound of $\mathbb{R}(\mathcal{S}, \mathcal{R})$ which can be computed by a convex optimization technique. We detail the upper bound estimation as follows. Consider a size-k seed set $S \in \mathcal{S}$. We denote the vector $(f_S(\mathcal{R}_1)), f_S(\mathcal{R}_2), ..., f_S(\mathcal{R}_l))$ by $\mathbf{v}(S, \mathcal{R})$. Then, let $\mathcal{V}(\mathcal{R})$ denote the set $\{\mathbf{v}(S, \mathcal{R}) | S \in \mathcal{S}\}$. The following lemma provides an upper bound of $\mathbb{R}(\mathcal{S}, \mathcal{R})$ which is a key component in the empirical error $\mathcal{E}$.

Theorem 5. (*Massart's Lemma, Thm. 3.3 [4]*).

$$\mathbb{R}(\mathcal{S}, \mathcal{R}) \leq \max_{S \in \mathcal{S}} |v(S, \mathcal{R})| \cdot \frac{\sqrt{2 \ln |\mathcal{V}(R)|}}{|\mathcal{R}|} \tag{12}$$

Although the above is the form in which the Theorem is usually stated, a careful reading of its proof allows us to state the following stronger version.

Theorem 6. *Consider the functions $w : \mathbf{R}^+ \to \mathbf{R}^+$ which is defined as follows.*

$$w(x) = \frac{1}{x} \cdot \ln(|\mathcal{V}(\mathcal{R})| \cdot \exp(\frac{x^2 \cdot \tilde{Pr}[S^* \cap R_r \neq \emptyset]}{2 \cdot l})) \tag{13}$$

Then, we have

$$\mathbb{R}(\mathcal{S}, \mathcal{R}) \leq \min_{x \in \mathbf{R}^+} w(x) \tag{14}$$

Proof. For any strictly positive variable x, we have

$$\begin{aligned} & e^{x \cdot \mathbb{R}(\mathcal{S}, \mathcal{R})} = e^{x \cdot \mathrm{E}_\sigma[\sup_{S \in \mathcal{S}}(\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i))]} \\ \leq & \mathrm{E}_\sigma[e^{x \cdot \sup_{S \in \mathcal{S}}(\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i))}] \leq \sum_{S \in \mathcal{S}} \mathrm{E}_\sigma[e^{x \cdot (\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i))}] \end{aligned}$$

By Theorem 3.3 of [4], we have

$$\mathrm{E}_\sigma[e^{x \cdot (\frac{1}{l}\sum_{i=1}^{l} \sigma_i \cdot f_S(\mathcal{R}_i))}] \leq e^{\frac{x^2 \cdot ||\mathbf{v}(S,\mathcal{R})||^2}{2 \cdot l^2}}$$

By the two inequalities above, we obtain that

$$\begin{aligned} & e^{x \cdot \mathbb{R}(\mathcal{S}, \mathcal{R})} \leq \sum_{S \in \mathcal{S}} e^{\frac{x^2 \cdot ||\mathbf{v}(S,\mathcal{R})||^2}{2 \cdot l^2}} \leq |\mathcal{V}(\mathcal{R})| \cdot \max_{\mathbf{v} \in \mathcal{V}(\mathcal{R})} e^{\frac{x^2 \cdot ||\mathbf{v}||^2}{2 \cdot l^2}} \\ \leq & |\mathcal{V}(\mathcal{R})| \cdot e^{\frac{x^2 \cdot ||\mathbf{v}(S^*,\mathcal{R})||^2}{2 \cdot l^2}} = |\mathcal{V}(\mathcal{R})| \cdot e^{\frac{x^2 \cdot \tilde{Pr}[S^* \cap R_r \neq \emptyset]}{2 \cdot l}} \end{aligned}$$

Now, it suffices to take the logarithm on both sides and divide by x (which is strictly positive). □

Let ω^* denote $\min_{s \in \mathbf{R}^+} w(x)$. Since this function $w(x)$ is convex, continuous and has first and second derivatives w.r.t. s everywhere in its domain, so we adopt the standard convex optimization methods [5] to find its minimum value ω^*. By Theorem 2 and Theorem 6, we obtain that

$$\mathcal{E} \leq 2 \cdot \omega^* + \frac{\ln \frac{2}{\eta}}{2l} \tag{15}$$

We denote the right hand side of Inequality (15) by $\mathcal{E}^+$ which means the upper bound of $\mathcal{E}$. By Theorem 4 and Inequality (15), we obtain that

$$\begin{aligned} \frac{\mathbb{I}_{\mathbb{C}}(S*)}{\mathbb{I}_{\mathbb{C}}(S^{OPT})} \geq & \frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1-\frac{1}{e}) + \mathcal{E}} \\ \geq & \frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}^+}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1-\frac{1}{e}) + \mathcal{E}^+} \end{aligned} \tag{16}$$

Finally, we obtain the following theorem which serves as our method for estimating the approximation ratio.

Theorem 7. *With the probability at least $1-\eta$, the approximation ratio of our algorithm is at least $\frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}^+}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1-\frac{1}{e}) + \mathcal{E}^+}$.*

Our algorithm of approximation ratio estimation is shown in Algorithm 3. It first computes ω^* from Line 1 to Line 3 and also computes $\mathcal{E}^+$ in Line 4 and finally returns the approximation ratio estimated by using Theorem 7 together with ω^*.

4.3 Sampling Schedule

Since our desired approximation ratio is $1-\frac{1}{e}-\epsilon$, in each iteration, we strive to achieve this desired ratio by setting the sample size properly. To this end, we first derive the corresponding $\mathcal{E}^+$ as follows.

Theorem 8. *The approximation ratio of our algorithm is* $1-\frac{1}{e}-\epsilon$ *iff* $\mathcal{E}^+$ *is at most* $\tilde{Pr}[S^* \cap R_r \neq \emptyset]/[\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e})]$.

Proof. By Theorem 7, we obtain that

$$\frac{\mathbb{I}_{\mathbb{C}}(S*)}{\mathbb{I}_{\mathbb{C}}(S^{OPT})} \geq \frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset] - \mathcal{E}^+}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/(1-\frac{1}{e}) + \mathcal{E}^+} = 1 - \frac{1}{e} - \frac{(1-\frac{1}{e})^2 + (1-\frac{1}{e})}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/\mathcal{E}^+ + (1-\frac{1}{e})} \tag{17}$$

Algorithm 4 getSampleSize($\mathcal{R}$, S^*, ω^*, ϵ, η, l)

Input: A set $\mathcal{R}$ of RR sets, a size-k seed set S^*, a real-valued number ω^*, an error parameter ϵ, a failure probability threshold η and the total sample size l

Output: The sample size Δl for the next iteration

1: $\tilde{Pr}[S^* \cap R_r \neq \emptyset] \leftarrow \frac{1}{|\mathcal{R}|}\sum_{R\in\mathcal{R}} f_S(R)$;

2: $l' \leftarrow \ln\frac{2}{\eta}/(2\cdot\frac{\tilde{Pr}[S^*\cap R_r\neq\emptyset]}{\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon}-(1-\frac{1}{e})} - 4\cdot\omega^*)$;

3: Return $l' - l$;

Then, it suffices to prove that $\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\tilde{Pr}[S^*\cap R_r\neq\emptyset]/\mathcal{E}^+ + (1-\frac{1}{e})} \leq \epsilon$ iff $\mathcal{E}^+$ is at most $\tilde{Pr}[S^* \cap R_r \neq \emptyset]/[\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e})]$.

$$\begin{aligned} &\frac{(1-\frac{1}{e})^2 + (1-\frac{1}{e})}{\tilde{Pr}[S^* \cap R_r \neq \emptyset]/\mathcal{E}^+ + (1-\frac{1}{e})} \leq \epsilon \\ \Leftrightarrow &\tilde{Pr}[S^* \cap R_r \neq \emptyset]/\mathcal{E}^+ \geq \frac{(1-\frac{1}{e})^2 + (1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e}) \\ \Leftrightarrow &\mathcal{E}^+ \leq \tilde{Pr}[S^* \cap R_r \neq \emptyset]/[\frac{(1-\frac{1}{e})^2 + (1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e})] \end{aligned} \tag{18}$$

□

Recall that $\mathcal{E}^+$ is equal to $2\cdot\omega^* + \frac{\ln\frac{2}{\eta}}{2l}$ and we derive the sample size by solving the following inequality.

$$\mathcal{E}^+ = 2\cdot\omega^* + \frac{\ln\frac{2}{\eta}}{2l} \leq \tilde{Pr}[S^*\cap R_r \neq \emptyset]/[\frac{(1-\frac{1}{e})^2 + (1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e})]$$

Thus, we obtain that

$$l \geq \ln\frac{2}{\eta}/(2\cdot\frac{\tilde{Pr}[S^* \cap R_r \neq \emptyset]}{\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon} - (1-\frac{1}{e})} - 4\cdot\omega^*) \tag{19}$$

In each iteration, we make the conservative estimation of the sample size, that is, by Inequality (19), the smallest sample size which can guarantee the desired error bound (i.e., we set the total sample size to be $l' = \ln\frac{2}{\eta}/(2\cdot\frac{\tilde{Pr}[S^*\cap R_r\neq\emptyset]}{\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon}-(1-\frac{1}{e})} - 4\cdot\omega^*)$). It is worth mentioning that when we calculate the initial sample size in Line 2 of Algorithm 2, we do not have the information with regard to S^* and ω^*, we keep using the optimistic (i.e., conservative) estimation in which we assume that $\tilde{Pr}[S^* \cap R_r \neq \emptyset] = 1$ and $\omega^* = 0$. As such, our initial sample size is set to be $\ln\frac{2}{\eta}/(2\cdot\frac{1}{\frac{(1-\frac{1}{e})^2+(1-\frac{1}{e})}{\epsilon}-(1-\frac{1}{e})})$. Our algorithm of sample size estimation is shown in Algorithm 4 and the returned Δl is the difference between l' and l.

Finally, we obtain the following theorem.

Theorem 9. *Given a failure probability threshold* $\eta \in (0,1)$ *and an error parameter* $\epsilon \in (0,1)$*, our algorithm returns an* $(1-\frac{1}{e}-\epsilon)$*-approximate solution for IM with the probability at least* $1-\eta$.

5 EXPERIMENT

5.1 Experimental Setting

We conducted our experiments on a Linux machine with 2.67 GHz CPU and 32GB memory. All algorithms were implemented in C++.

Datasets. Following the existing study [34] on OPIM, we adopted four real datasets in the experiment, namely, Pokec, Orkut, LiveJournal, and Twitter. Table 1 shows the statistics of each dataset. Each of these datasets is collected from a social network and could be downloaded at this link[1].

Dataset	n	m	Type	Avg. degree
Pokec	1.6M	30.6M	directed	37.5
Orkut	3.1M	117.2M	undirected	76.3
LiveJournal	4.8M	69.0M	directed	28.5
Twitter	41.7M	1.5G	directed	70.5

Table 1: Statistics of the Datasets (K = 10^3, M = 10^6, G = 10^9)

Algorithms. Following the state-of-the-art work [21] in OPIM, we tested our algorithm, OPIM-C [34] and SUBSIM [20, 21] algorithms in the experiment besides our proposed algorithm. We safely ignore other existing algorithms since any other algorithm was proven to be inferior to OPIM-C and SUBSIM according to the empirical study of [20, 21]. It is worth mentioning that the technique of HIST proposed in [20, 21] is integrated in SUBSIM to boost its performance. We tested the performance of each algorithm under both IC and LT models. Following [20, 21], we study IC model with four different distribution settings, namely *WC*, *Exponential Distribution*, *Weibull Distribution* and *Uniform*. We refer the readers to [20, 21] for the detailed description of each definition for the sake of limited space.

Factors & Measurements. Following [34], we studied two factors in our experiments: (1) the size k of the seed set, and (2) the error parameter ϵ. By default, we set the error parameter $\epsilon = 0.01$, the failure probability threshold $\eta = \frac{1}{n}$ and the size of the seed set $k = 50$. Three measurements, namely (1) the number of RR sets generated, (2) the total running time and (3) the influence of returned size-k seed set, are considered in the experiment.

5.2 Experimental Results

Effect of k. We studied the effect of k on the performances of all algorithms. We tested six different values of k: 1, 10, 50, 100, 150, 200. Figure 1, Figure 2 and Figure 3 demonstrate our study on the WC setting of the IC model. They show the running time, the No. RR sets sampled and the influence

[1] https://snap.stanford.edu/data/

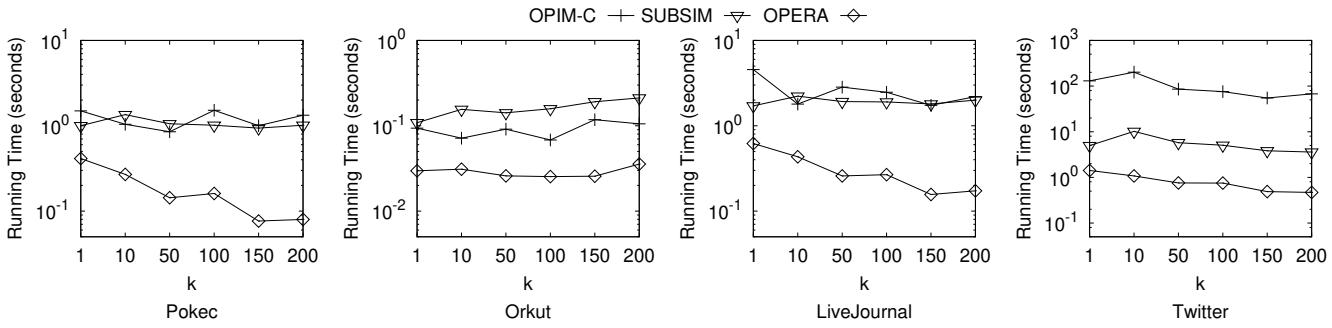

Figure 1: Effect of k on Running Time of All Algorithms under IC Model (WC)

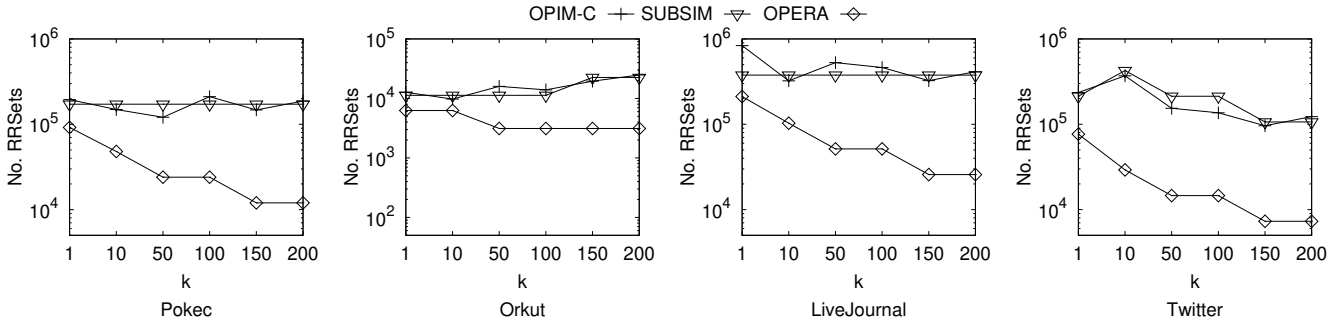

Figure 2: Effect of k on No. RR sets of All Algorithms under IC Model (WC)

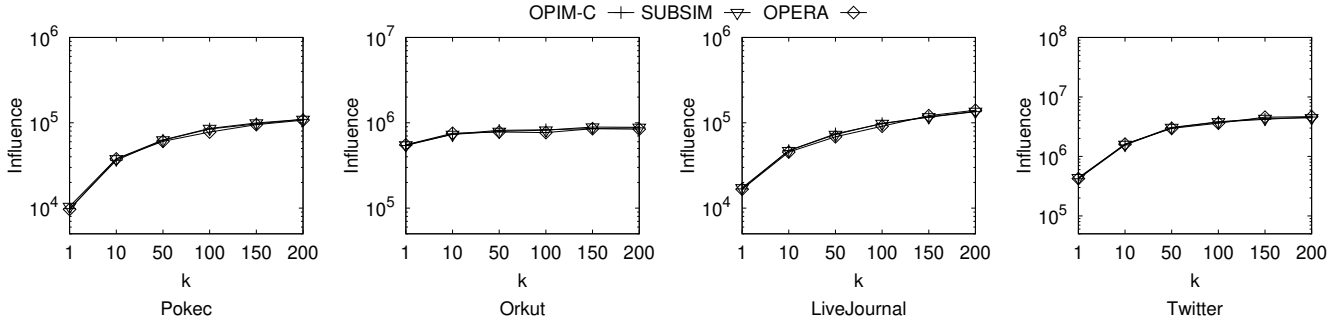

Figure 3: Effect of k on Influences of All Algorithms under IC Model (WC)

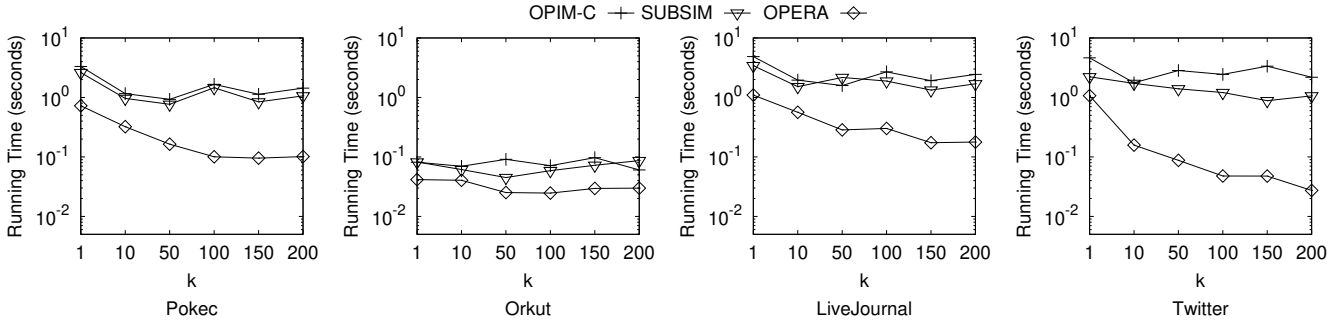

Figure 4: Effect of k on Running Time of All Algorithms under IC Model (Exponential Distribution)

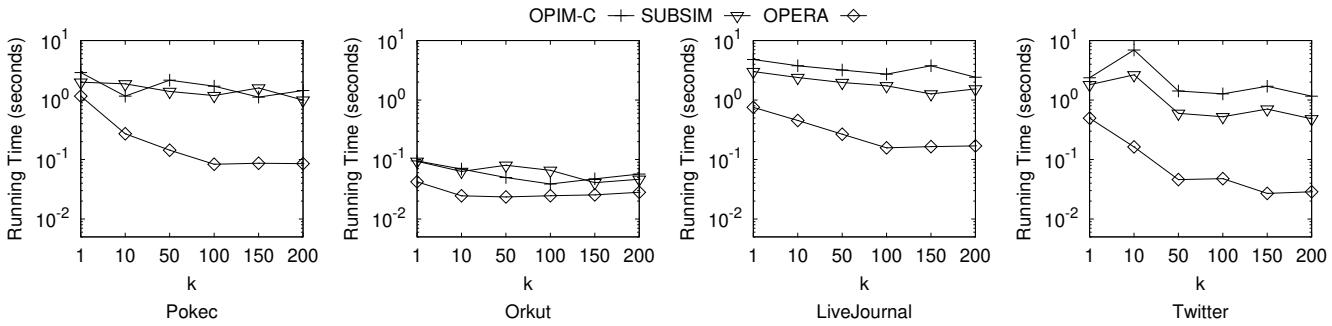

Figure 5: Effect of k on Running Time of All Algorithms under IC Model (Weibull Distribution)

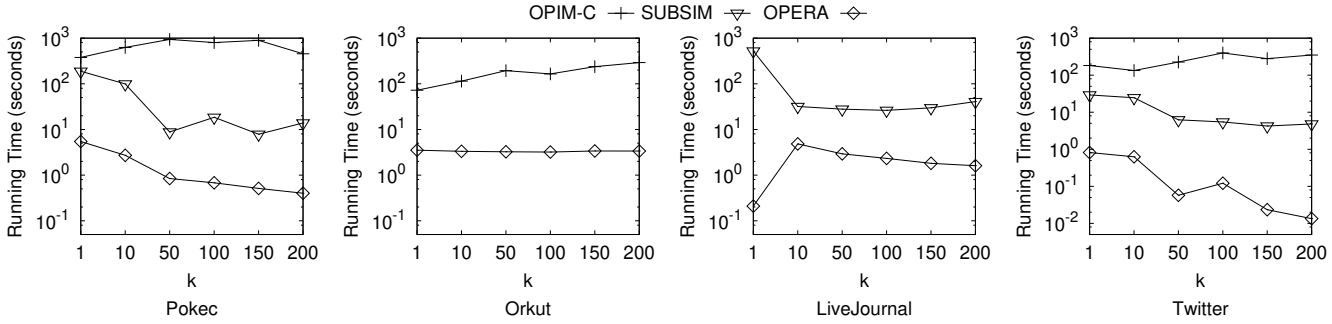

Figure 6: Effect of k on Running Time of All Algorithms under IC Model (Uniform Distribution)

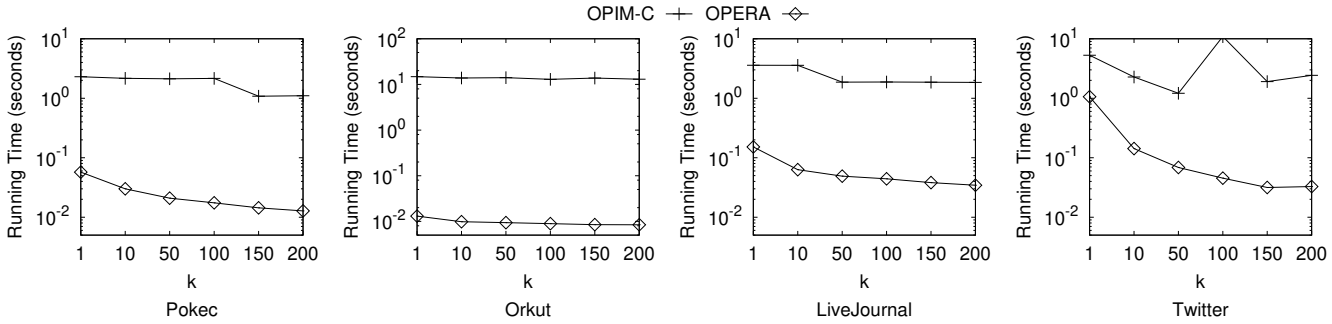

Figure 7: Effect of k on Running Time of All Algorithms under LT Model

of the returned seed set for each algorithm considered under the five values of k. As the figures show, our algorithm outperforms the two state-of-the-art algorithms by up to more than one order of magnitudes in terms of running time and No. RR sets with nearly the same influence obtained. The result confirms that our "killing two birds with one stone" technique really help reduce the number of samples and boosts the sampling efficiency. We also studied the effect of k under the exponential distribution, Weibull distribution and the uniform distribution of the IC model and the LT model. Figure 4, Figure 5, Figure 6 and Figure 7 show the running time of all algorithms tested under the four models. Note that SUBSIM only applies for IC model and does not apply for LT model and thus, we only compare our algorithm with OPIM-C in Figure 7. Similar results were observed in the four figures.

Effect of ϵ. We studied the effect of ϵ on the performances of all algorithms. We tested six different values of ϵ: 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09. Figure 8 demonstrates our study on the WC setting of the IC model. They show the running time of the returned seed set for each algorithm considered under the different values of ϵ. As the figures show, our algorithm outperforms the two state-of-the-art algorithms by

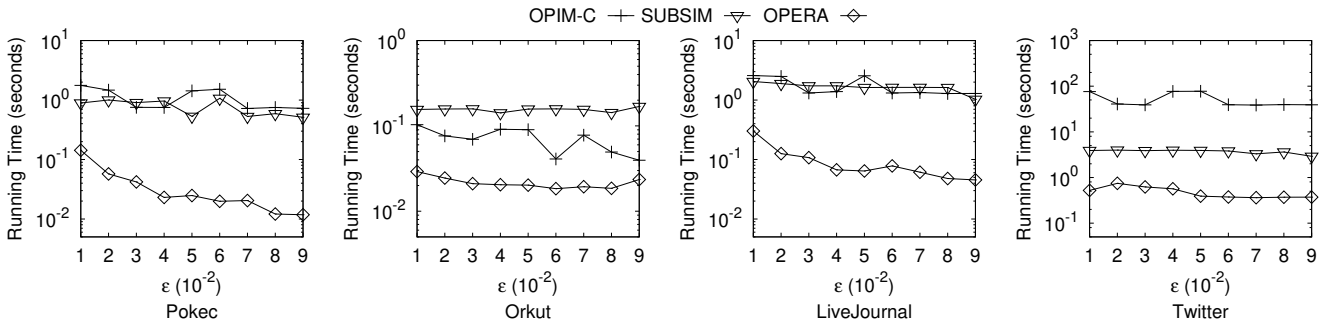

Figure 8: Effect of ϵ on Running Time of All Algorithms under IC Model (WC)

up to more than one order of magnitudes in terms of running time and No. RR sets with nearly the same influence obtained. The result confirms that our technique really help reduce the number of samples and boosts the sampling efficiency. In a word, our algorithm generates a significantly smaller number of RR sets compared with the baselines while achieves almost the same influence. We also studied the effect of ϵ under the exponential distribution, Weibull distribution and the uniform distribution of the IC model and the LT model. Figure 11, Figure 12, Figure 13 and Figure 14 show the running time of all algorithms tested under the four models. Note that SUBSIM only applies for IC model and does not apply for LT model and thus, we only compare our algorithm with OPIM-C in Figure 14. Similar results were observed in the four settings.

5.3 Experimental Result Summary

In the experiment, we tested our algorithm and the existing state-of-the-art algorithms and studies the effects of two important factors k and ϵ. We observe that our algorithm outperforms all existing algorithms by up to more than one order of magnitudes in terms of sample size and running time.

6 CONCLUSION

In this paper, we propose a novel algorithm *OPERA* for the online processing of influence maximization problem iteratively in an online fashion. The key feature of our algorithm is "killing two birds with one stone" which in each iteration finds the seed set and estimate the currently achieved quality assurance by using one set of samples. Besides, we also propose an error-aware dynamic sampling schedule method. Our quality estimation and the sampling schedule is based on a concept called *Rademacher Average* from statistical learning theory. As such, we highly boost the sampling efficiency while achieving the same approximation guarantee with the state-of-the-art algorithm in theory and our empirical study shows that our algorithms outperforms all existing algorithms by up to more than one order of magnitudes in terms of sample size and running time.

REFERENCES

[1] Akhil Arora, Sainyam Galhotra, and Sayan Ranu. 2017. Debunking the myths of influence maximization: An in-depth benchmarking study. In *Proceedings of the 2017 ACM international conference on management of data*. 651–666.

[2] Glenn S Bevilacqua and Laks VS Lakshmanan. 2022. A fractional memory-efficient approach for online continuous-time influence maximization. *The VLDB Journal* 31, 2 (2022), 403–429.

[3] Christian Borgs, Michael Brautbar, Jennifer Chayes, and Brendan Lucier. 2014. Maximizing social influence in nearly optimal time. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 946–957.

[4] Stéphane Boucheron, Olivier Bousquet, and Gábor Lugosi. 2005. Theory of classification: A survey of some recent advances. *ESAIM: probability and statistics* (2005).

[5] Stephen Boyd, Stephen P Boyd, and Lieven Vandenberghe. 2004. *Convex optimization*. Cambridge university press.

[6] Shuo Chen, Ju Fan, Guoliang Li, Jianhua Feng, Kian-lee Tan, and Jinhui Tang. 2015. Online topic-aware influence maximization. *Proceedings of the VLDB Endowment* 8, 6 (2015), 666–677.

[7] Wei Chen, Chi Wang, and Yajun Wang. 2010. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1029–1038.

[8] Wei Chen, Yajun Wang, and Siyu Yang. 2009. Efficient influence maximization in social networks. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. 199–208.

[9] Wei Chen, Yifei Yuan, and Li Zhang. 2010. Scalable influence maximization in social networks under the linear threshold model. In *2010 IEEE international conference on data mining*. IEEE, 88–97.

[10] Xuanhao Chen, Yan Zhao, Guanfeng Liu, Rui Sun, Xiaofang Zhou, and Kai Zheng. 2020. Efficient similarity-aware influence maximization in geo-social network. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4767–4780.

[11] Suqi Cheng, Huawei Shen, Junming Huang, Wei Chen, and Xueqi Cheng. 2014. IMRank: influence maximization via finding self-consistent ranking. In *Proceedings of the 37th international ACM SIGIR conference on Research & development in information retrieval*. 475–484.

[12] Suqi Cheng, Huawei Shen, Junming Huang, Guoqing Zhang, and Xueqi Cheng. 2013. Staticgreedy: solving the scalability-accuracy dilemma in influence maximization. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*. 509–518.

[13] Edith Cohen, Daniel Delling, Thomas Pajor, and Renato F Werneck. 2014. Sketch-based influence maximization and computation: Scaling up with guarantees. In *Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management*. 629–638.

[14] Pedro Domingos and Matt Richardson. 2001. Mining the network value of customers. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 57–66.

[15] Sainyam Galhotra, Akhil Arora, and Shourya Roy. 2016. Holistic influence maximization: Combining scalability and efficiency with opinion-aware models. In *Proceedings of the 2016 International Conference on Management of Data*. 743–758.

[16] Sainyam Galhotra, Akhil Arora, Srinivas Virinchi, and Shourya Roy. 2015. Asim: A scalable algorithm for influence maximization under the independent cascade model. In *Proceedings of the 24th International Conference on World Wide Web*. 35–36.

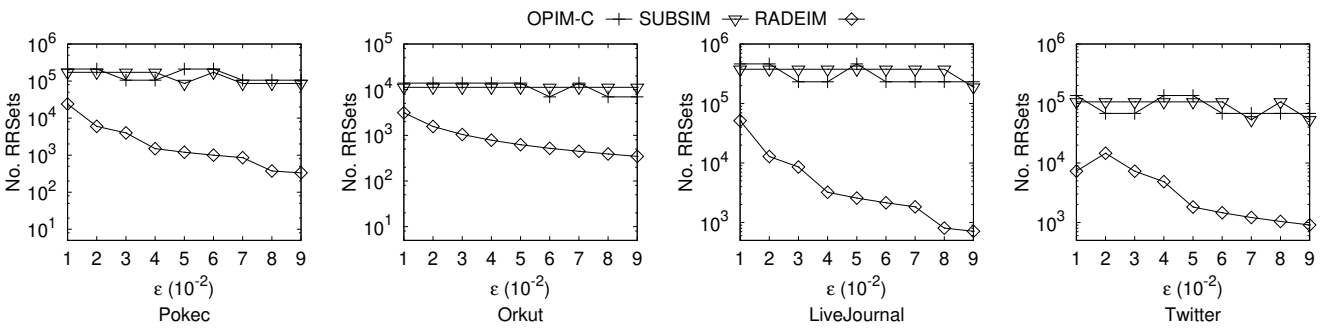

Figure 9: Effect of ϵ on No. RR sets of All Algorithms under IC Model (WC)

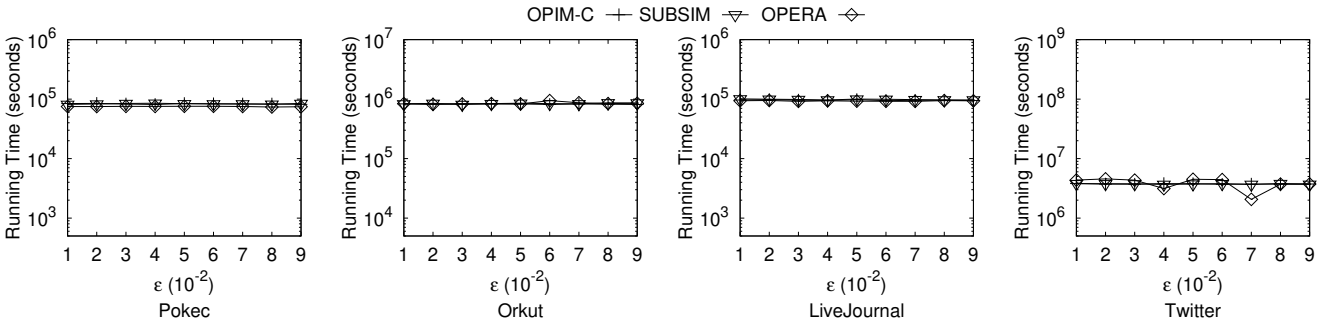

Figure 10: Effect of ϵ on Influence of All Algorithms under IC Model (WC)

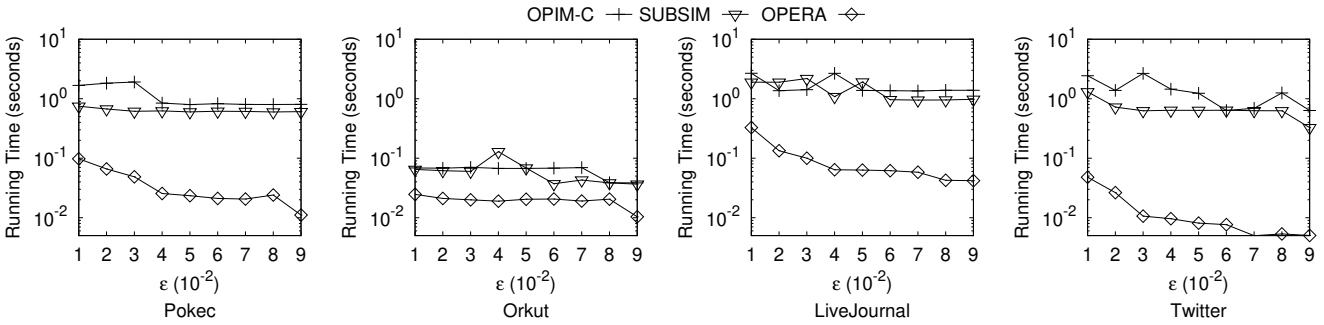

Figure 11: Effect of ϵ on Running Time of All Algorithms under IC Model (Exponential Distribution)

[17] Amit Goyal, Francesco Bonchi, and Laks VS Lakshmanan. 2011. A data-based approach to social influence maximization. *arXiv preprint arXiv:1109.6886* (2011).

[18] Amit Goyal, Wei Lu, and Laks VS Lakshmanan. 2011. Celf++ optimizing the greedy algorithm for influence maximization in social networks. In *Proceedings of the 20th international conference companion on World wide web*. 47–48.

[19] Amit Goyal, Wei Lu, and Laks VS Lakshmanan. 2011. Simpath: An efficient algorithm for influence maximization under the linear threshold model. In *2011 IEEE 11th international conference on data mining*. IEEE, 211–220.

[20] Qintian Guo, Sibo Wang, Zhewei Wei, and Ming Chen. 2020. Influence maximization revisited: Efficient reverse reachable set generation with bound tightened. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2167–2181.

[21] Qintian Guo, Sibo Wang, Zhewei Wei, Wenqing Lin, and Jing Tang. 2022. Influence Maximization Revisited: Efficient Sampling with Bound Tightened. *ACM Transactions on Database Systems (TODS)* (2022).

[22] Keke Huang, Sibo Wang, Glenn Bevilacqua, Xiaokui Xiao, and Laks VS Lakshmanan. 2017. Revisiting the stop-and-stare algorithms for influence maximization. *Proceedings of the VLDB Endowment* 10, 9 (2017), 913–924.

[23] Kyomin Jung, Wooram Heo, and Wei Chen. 2012. Irie: Scalable and robust influence maximization in social networks. In *2012 IEEE 12th international conference on data mining*. IEEE, 918–923.

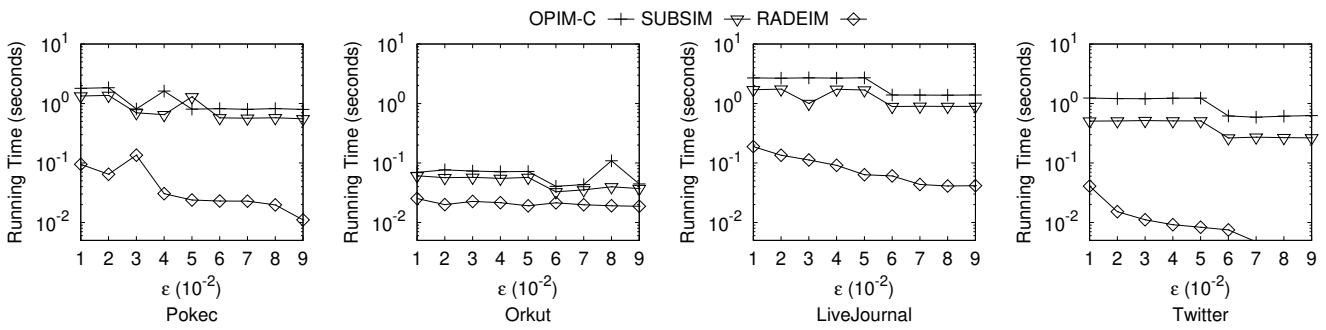

Figure 12: Effect of ϵ on Running Time of All Algorithms under IC Model (Weibull Distribution)

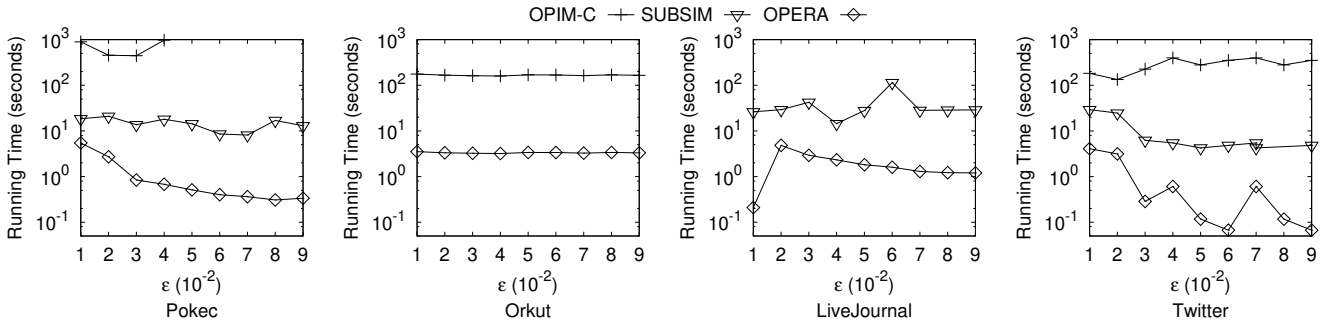

Figure 13: Effect of ϵ on Running Time of All Algorithms under IC Model (Uniform Distribution)

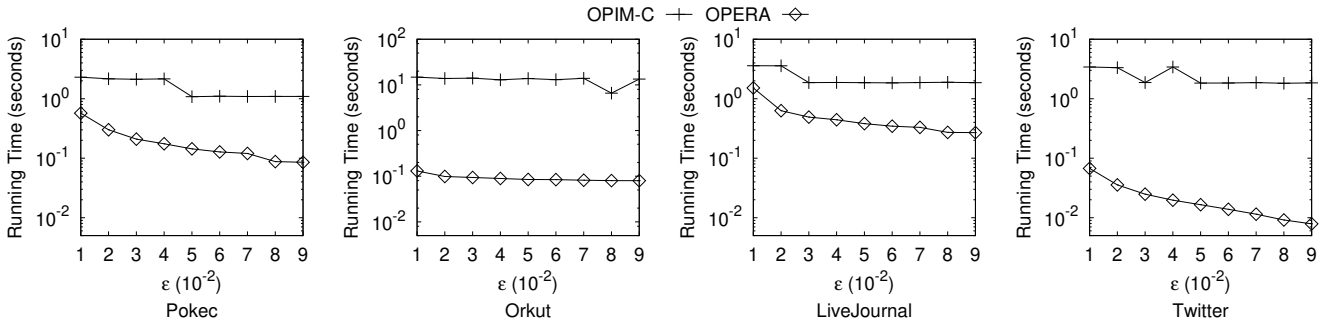

Figure 14: Effect of ϵ on Running Time of All Algorithms under LT Model

[24] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.

[25] Jong-Ryul Lee and Chin-Wan Chung. 2014. A fast approximation for influence maximization in large social networks. In *Proceedings of the 23rd international conference on World Wide Web*. 1157–1162.

[26] Siyu Lei, Silviu Maniu, Luyi Mo, Reynold Cheng, and Pierre Senellart. 2015. Online influence maximization. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 645–654.

[27] Yuchen Li, Dongxiang Zhang, and Kian-Lee Tan. 2015. Real-time targeted influence maximization for online advertisements. (2015).

[28] Hung T Nguyen, My T Thai, and Thang N Dinh. 2016. Stop-and-stare: Optimal sampling algorithms for viral marketing in billion-scale networks. In *Proceedings of the 2016 international conference on management of data*. 695–710.

[29] Naoto Ohsaka, Takuya Akiba, Yuichi Yoshida, and Ken-ichi Kawarabayashi. 2014. Fast and accurate influence maximization on large networks with pruned monte-carlo simulations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 28.

[30] Jeff John Roberts. 2016. Facebook and Google are big winners as political ad money moves online. *Fortune, July* 21 (2016).

[31] Stephan R Sain. 1996. The nature of statistical learning theory.

[32] Shai Shalev-Shwartz and Shai Ben-David. 2014. *Understanding machine learning: From theory to algorithms*. Cambridge university press.
[33] Guojie Song, Xiabing Zhou, Yu Wang, and Kunqing Xie. 2014. Influence maximization on large-scale mobile social network: a divide-and-conquer method. *IEEE Transactions on Parallel and Distributed Systems* 26, 5 (2014), 1379–1392.
[34] Jing Tang, Xueyan Tang, Xiaokui Xiao, and Junsong Yuan. 2018. Online processing algorithms for influence maximization. In *Proceedings of the 2018 International Conference on Management of Data*. 991–1005.
[35] Jing Tang, Xueyan Tang, and Junsong Yuan. 2016. Profit maximization for viral marketing in online social networks. In *2016 IEEE 24th International Conference on Network Protocols (ICNP)*. IEEE, 1–10.
[36] Jing Tang, Xueyan Tang, and Junsong Yuan. 2017. Influence maximization meets efficiency and effectiveness: A hop-based approach. In *2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 64–71.
[37] Jing Tang, Xueyan Tang, and Junsong Yuan. 2017. Profit maximization for viral marketing in online social networks: Algorithms and analysis. *IEEE Transactions on Knowledge and Data Engineering* 30, 6 (2017), 1095–1108.
[38] Jing Tang, Xueyan Tang, and Junsong Yuan. 2018. An efficient and effective hop-based approach for influence maximization in social networks. *Social Network Analysis and Mining* 8, 1 (2018), 1–19.
[39] Jing Tang, Xueyan Tang, and Junsong Yuan. 2018. Towards profit maximization for online social network providers. In *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 1178–1186.
[40] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence maximization in near-linear time: A martingale approach. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 1539–1554.
[41] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: Near-optimal time complexity meets practical efficiency. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 75–86.
[42] Ya-Wen Teng, Yishuo Shi, Chih-Hua Tai, De-Nian Yang, Wang-Chien Lee, and Ming-Syan Chen. 2021. Influence maximization based on dynamic personal perception in knowledge graph. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1488–1499.
[43] Vladimir N Vapnik. 1999. An overview of statistical learning theory. *IEEE transactions on neural networks* 10, 5 (1999), 988–999.
[44] Yanhao Wang, Qi Fan, Yuchen Li, and Kian-Lee Tan. 2017. Real-time influence maximization on dynamic social streams. *arXiv preprint arXiv:1702.01586* (2017).
[45] Yiming Ying. 2004. McDiarmid's inequalities of Bernstein and Bennett forms. (2004).
[46] Chuan Zhou, Peng Zhang, Jing Guo, and Li Guo. 2014. An upper bound based greedy algorithm for mining top-k influential nodes in social networks. In *Proceedings of the 23rd International Conference on World Wide Web*. 421–422.
[47] Chuan Zhou, Peng Zhang, Jing Guo, Xingquan Zhu, and Li Guo. 2013. Ublf: An upper bound based approach to discover influential nodes in social networks. In *2013 IEEE 13th International Conference on Data Mining*. IEEE, 907–916.

A ADDITIONAL EXPERIMENTS

Effect of ϵ with A Wider Range. We study the effect of the error parameter ϵ with a wider range which is up to 0.5 under the WC model. We tested the seven values: 0.01, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5. Figure 15, Figure 16 and Figure 17 show the running time, the number of RR sets sampled and the influence spread of each algorithm considered under these error parameter. Since we tested a wider range of ϵ in this experiment, we can observe that the running time and the no. of RR sets generated of each algorithm is obviously decreasing with the increase of ϵ. This is quite expected since the larger the ϵ is, the less stringent we require for the quality and as such, the less number of RR sets is required. As the figures reveals, *OPERA* outperforms the two baselines algorithms by up to one order of magnitudes in terms of the running time and the number of RR sets sampled with nearly the same influence.

Effect of the Failure Probability η. We study the effect of the failure probability η under the WC model. We tested the nine values: $10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}, 10^{-7}, 10^{-8}, 10^{-9}$. Figure 18, Figure 19 and Figure 20 show the running time, the number of RR sets sampled and the influence spread of each algorithm considered under these failure probabilities. The running time, the number of RR sets sampled and the influence spread of each algorithm is monotonically increasing with the decrease of η. This is because a smaller failure probability leads to a more stringent quality guarantee and requires more RR sets. As the figures reveals, *OPERA* outperforms the two baselines algorithms by up to one order of magnitudes in terms of the running time and the number of RR sets sampled with nearly the same influence.

Influence Spread VS Running Time & Appro. Ratio VS No. of RR Sets. We also tested the quality achieved of each algorithm over the sampling process to evaluate their online performances on Twitter dataset. Figure 21 demonstrates the influence spread VS running time and the approximation ratio VS No. of RR Sets. As the figures shows, OPERA achieves the same quality as the two baselines by using significantly smaller number of RR Sets and highly boosts the efficiency of the online IM processing.

Effect of k with A Wider Range. We study the effect of the error parameter k with a wider range which is up to 2000 under the WC model. We tested the six values: 1, 100, 500, 1000, 1500, 2000. Figure 22, Figure 23 and Figure 24 show the running time, the number of RR sets sampled and the influence spread of each algorithm considered under these error parameter. The running time and the no. of RR sets of each algorithm is in general decreasing with the increase of k since the larger k will make the seed set selection easier. The influence of each algorithm is in general increasing with the increase of k since the selection of more seed will incur larger influence spread. This result is consistent with that in [20, 21]. As the figures reveals, *OPERA* outperforms the two baselines algorithms by up to one order of magnitudes in terms of the running time and the number of RR sets sampled with nearly the same influence.

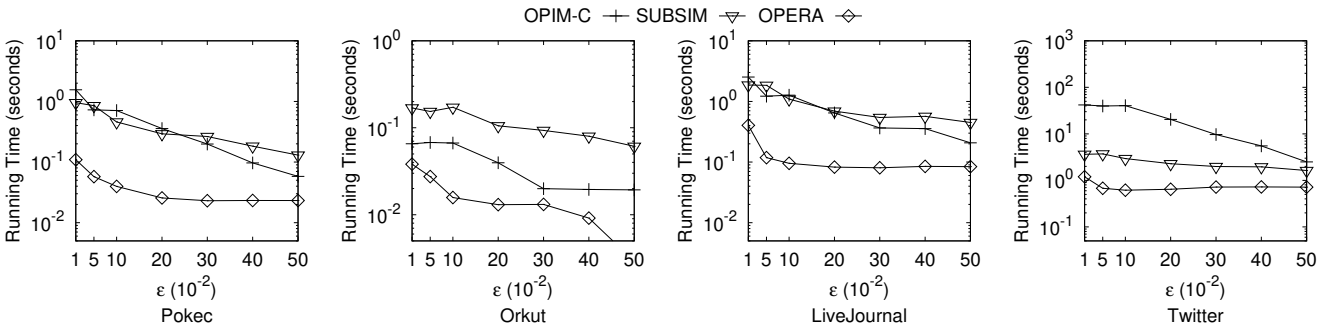

Figure 15: Effect of ϵ on Running Time of All Algorithms under IC Model (WC)

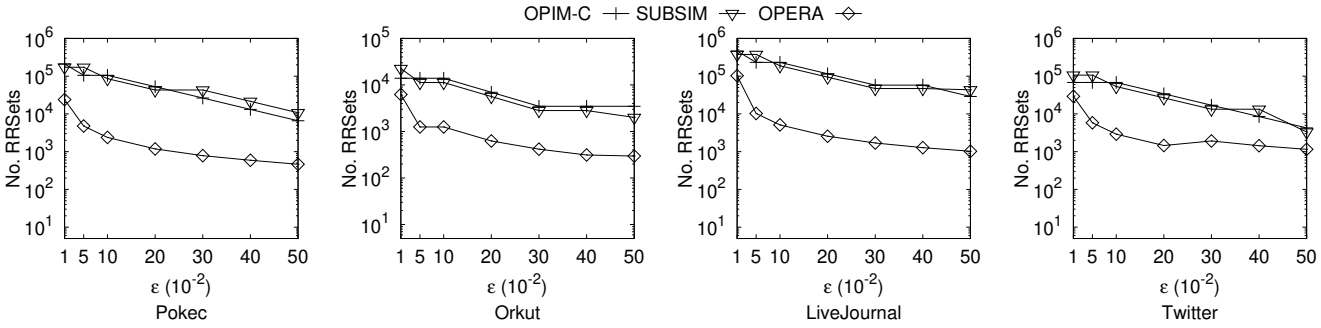

Figure 16: Effect of ϵ on No. RR sets of All Algorithms under IC Model (WC)

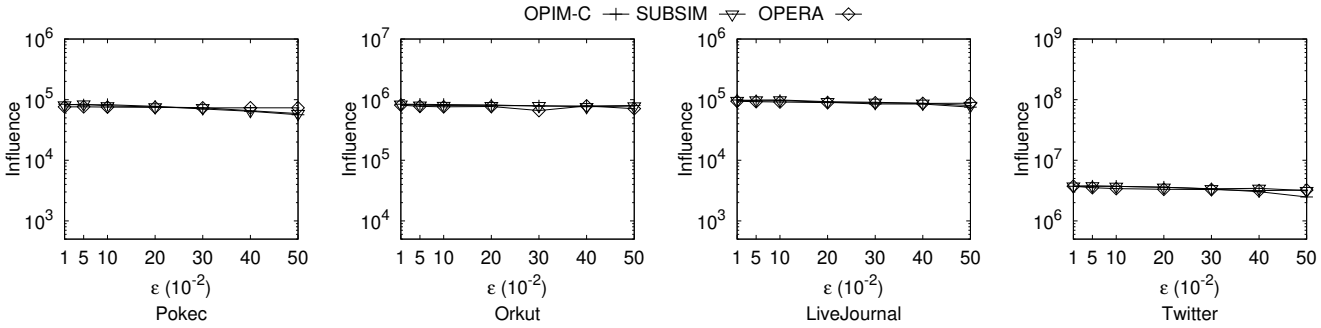

Figure 17: Effect of ϵ on Influence of All Algorithms under IC Model (WC)

B TIME COMPLEXITY OF OPERA

Theorem 10. *Given a failure probability $\eta \in (0,1)$ and an error parameter $\epsilon \in (0,1)$, the time complexity of OPERA to return a $(1-\frac{1}{e}-\epsilon)$-approximate seed set S^* with the probability at least $1-\eta$ is under WC model and Uniform IC model are $O(k \cdot n \cdot \log(\frac{1}{\eta})/\epsilon^2)$ and $O(k \cdot (m+n) \cdot \log(\frac{1}{\eta})/\epsilon^2)$, respectively.*

Proof. According to Lemma 3 of [40], we obtain that given a failure probability $\eta \in (0,1)$ and an error parameter $\epsilon \in (0,1)$, the number of RR sets, denoted by $N_{\mathcal{R}}$, required to ensure a $(1-\frac{1}{e}-\epsilon)$-approximate solution with probability at least $1-\eta$ can be bounded by $O(\frac{k \cdot n \cdot \log \frac{1}{\eta}}{\mathbb{I}_{\mathbb{C}}(S^{OPT}) \cdot \epsilon^2})$.

Consider the two IC models in the theorem. According to Lemma 4 and Theorem 1 of [21], we obtain that the time complexity of sampling one RR set, denoted by T_{R_r}, is $O(\mathbb{I}_{\mathbb{C}}(S^{OPT}))$ under WC model and is $O(\frac{m}{n} \cdot \mathbb{I}_{\mathbb{C}}(S^{OPT}))$ under Uniform IC model.

Then, we consider each iteration shown in Line 5-11 in Algorithm 2. The time complexity of the sampling of $\Delta\mathcal{R}$ (shown in Line 5) is $O(T_{R_r}) \cdot |\Delta\mathcal{R}|$. The operations in Line 5 invovles

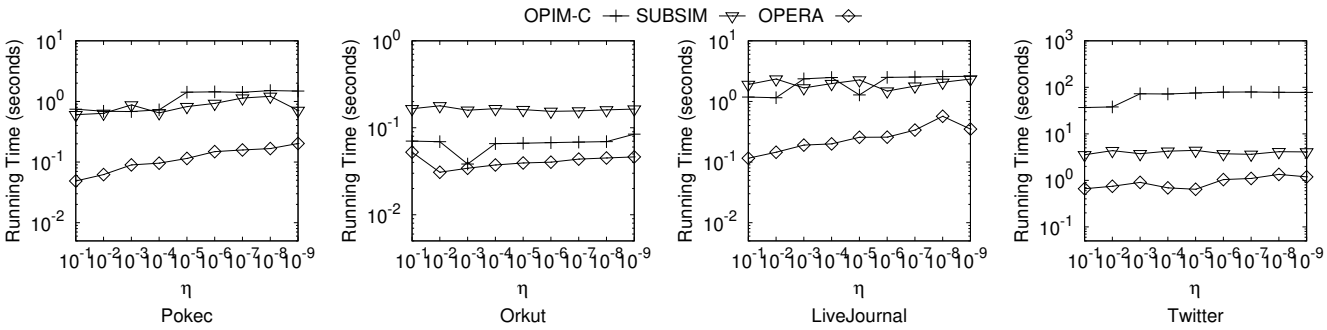

Figure 18: Effect of η on Running Time of All Algorithms under IC Model (WC)

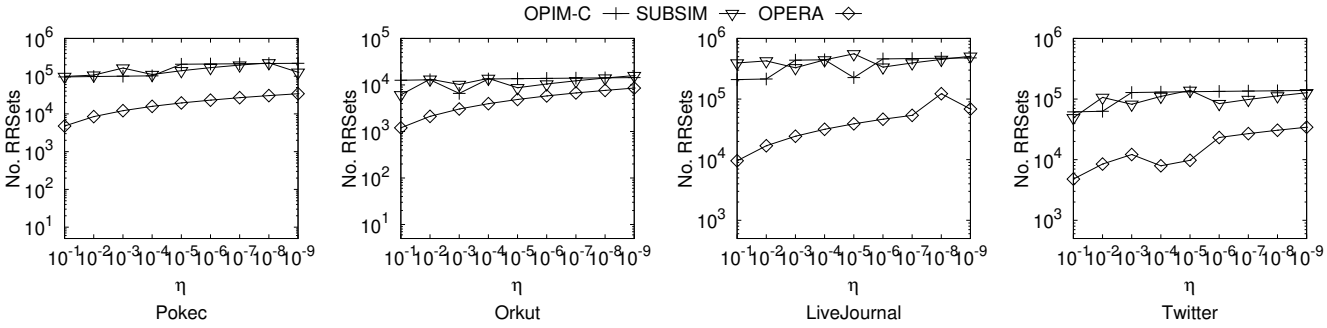

Figure 19: Effect of η on No. RR sets of All Algorithms under IC Model (WC)

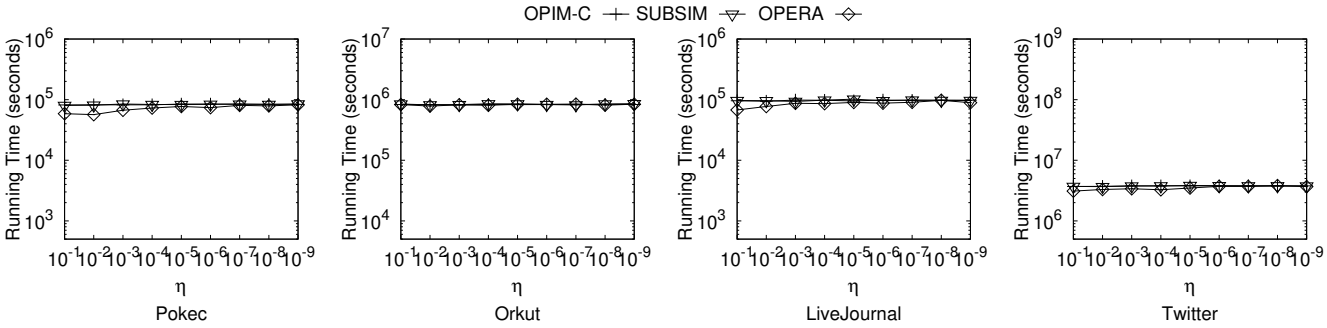

Figure 20: Effect of η on Influence of All Algorithms under IC Model (WC)

the standard set union and assignment operation and it takes $O(|\Delta\mathcal{R}|)$ time if we use a standard hash table to maintain $\mathcal{R}$. By [40], the time complexity of $greedyIM(\cdot)$ in Line 6 is $O(\sum_{R\in\Delta\mathcal{R}}|R|) = O(|\Delta\mathcal{R}|\cdot T_{R_r})$. Consider $geApproRatio(\cdot)$ in Line 7. The values of $\sum_{R\in\mathcal{R}} f_{S^*}(R)$ and $|\mathcal{V}(\mathcal{R})|$ are provided through $greedyIM(\cdot)$. In the convex optimization algorithm for the computation of ω^*, we set the maximum number of iterations to be $|\Delta\mathcal{R}|$ and adopt a constant batch size in each iteration. Thus, the time complexity of $geApproRatio(\cdot)$ in Line 7 is $O(|\Delta\mathcal{R}|)$. The time complexity of $getSampleSize(\cdot)$ in Line 11 is $O(1)$ since the values of $\sum_{R\in\mathcal{R}} f_{S^*}(R)$ is provided through $greedyIM(\cdot)$ and other operations in the algorithm takes constant time. Finally, we obtain that each iteration of OPERA takes $O(|\Delta\mathcal{R}|\cdot T_{R_r})$ time.

Thus, we obtain that the total running time of OPERA is upper bounded by $O(N_{\mathcal{R}}\cdot T_{R_r})$. This finishes the proof. □

Theorem 11. *Given a failure probability $\eta\in(0,1)$ and an error parameter $\epsilon\in(0,1)$, the time complexity of OPERA to return a $(1-\frac{1}{e}-\epsilon)$-approximate seed set S^* with the probability at least $1-\eta$ is under LT model is $O(k\cdot n\cdot\log(\frac{1}{\eta})/\epsilon^2)$.*

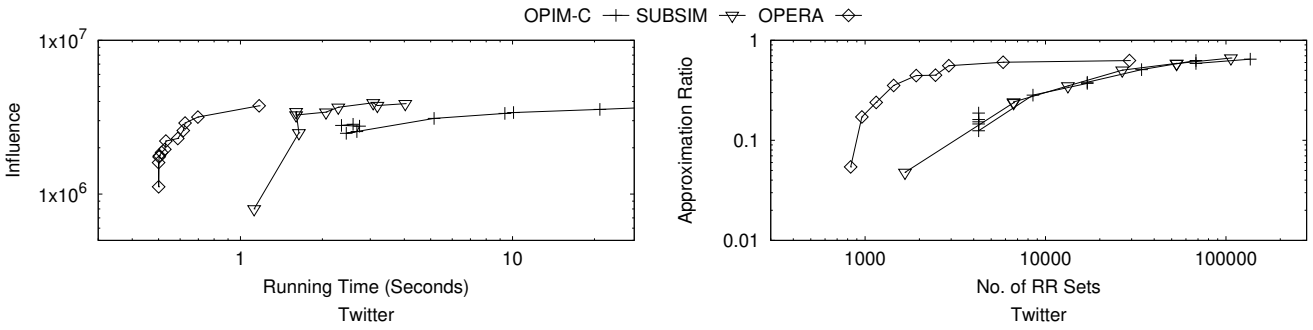

Figure 21: Influence Spread VS Running Time & Appro. Ratio VS No. of RR Sets

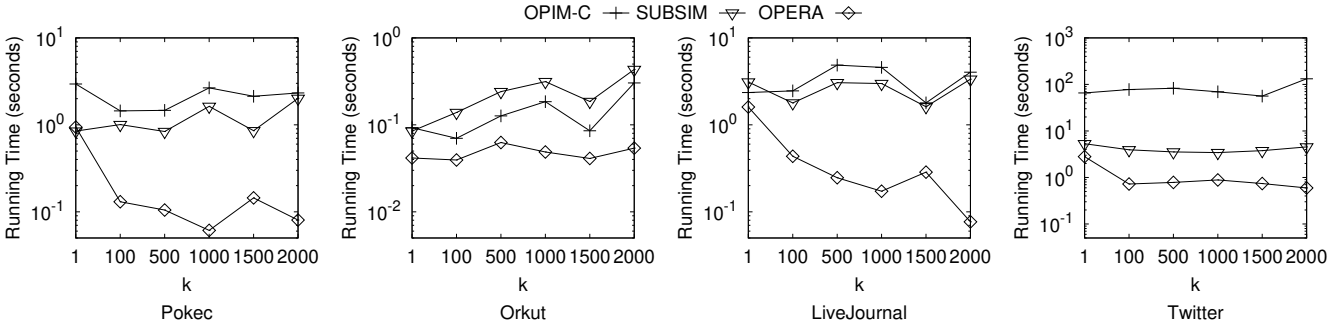

Figure 22: Effect of k on Running Time of All Algorithms under IC Model (WC)

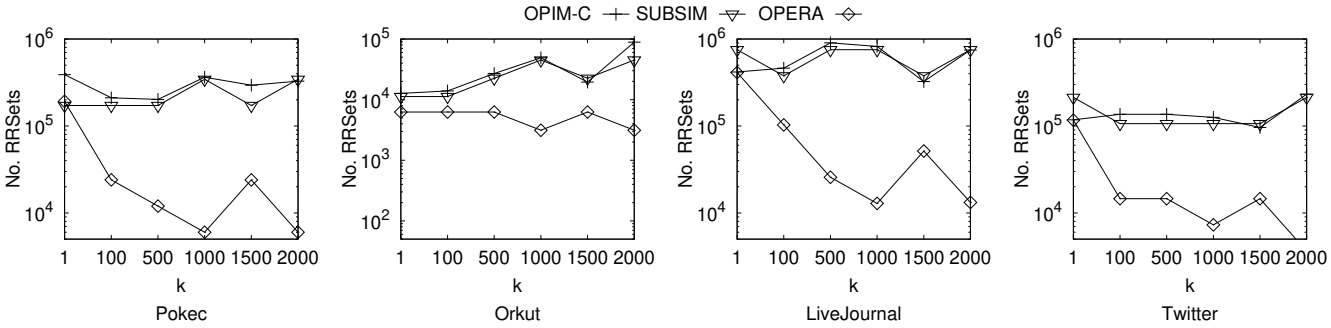

Figure 23: Effect of k on No. RR sets of All Algorithms under IC Model (WC)

Proof. According to Lemma 3 of [40], we obtain that given a failure probability $\eta \in (0,1)$ and an error parameter $\epsilon \in (0,1)$, the number of RR sets, denoted by $N_{\mathcal{R}}$, required to ensure a $(1-\frac{1}{e}-\epsilon)$-approximate solution with probability at least $1-\eta$ can be bounded by $O(\frac{k\cdot n\cdot \log\frac{1}{\eta}}{\mathbb{I}_{\mathbb{C}}(S^{OPT})\cdot\epsilon^2})$.

Consider the two IC models in the theorem. According to Lemma 4 and Theorem 1 of [21], we obtain that the time complexity of sampling one RR set, denoted by T_{R_r}, is $O(\mathbb{I}_{\mathbb{C}}(S^{OPT}))$ under WC model and is $O(\frac{m}{n}\cdot\mathbb{I}_{\mathbb{C}}(S^{OPT}))$ under Uniform IC model.

Then, we consider each iteration shown in Line 5-11 in Algorithm 2. The time complexity of the sampling of $\Delta\mathcal{R}$ (shown in Line 5) is $O(T_{R_r})\cdot|\Delta\mathcal{R}|$. The operations in Line 5 invovles the standard set union and assignment operation and it takes $O(|\Delta\mathcal{R}|)$ time if we use a standard hash table to maintain $\mathcal{R}$. By [40], the time complexity of $greedyIM(\cdot)$ in Line 6 is $O(\sum_{R\in\Delta\mathcal{R}}|R|) = O(|\Delta\mathcal{R}|\cdot T_{R_r})$. Consider $geApproRatio(\cdot)$ in Line 7. The values of $\sum_{R\in\mathcal{R}} f_{S^*}(R)$ and $|\mathcal{V}(\mathcal{R})|$ are provided through $greedyIM(\cdot)$. In the convex optimization algorithm for the computation of ω^*, we set the maximum number of iterations to be $|\Delta\mathcal{R}|$ and adopt a constant batch size in each iteration. Thus, the

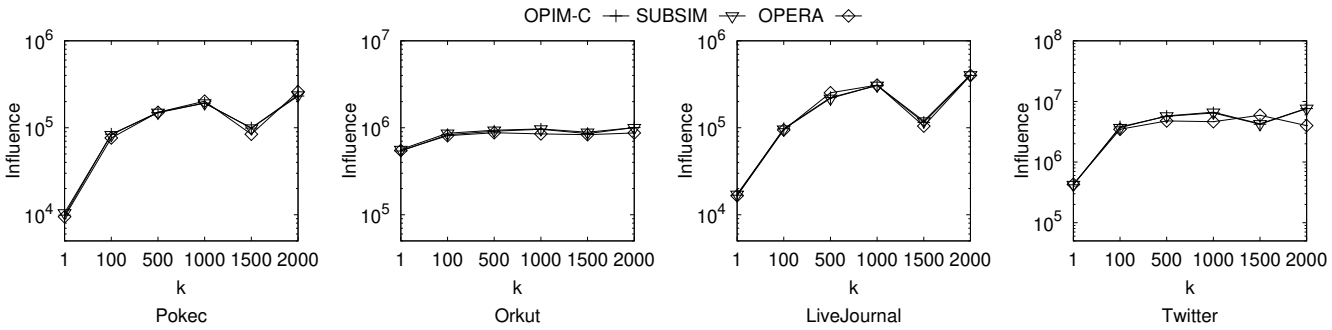

Figure 24: Effect of k on Influences of All Algorithms under IC Model (WC)

time complexity of $geApproRatio(\cdot)$ in Line 7 is $O(|\Delta\mathcal{R}|)$. The time complexity of $getSampleSize(\cdot)$ in Line 11 is $O(1)$ since the values of $\sum_{R\in\mathcal{R}} f_{S^*}(R)$ is provided through $greedyIM(\cdot)$ and other operations in the algorithm takes constant time. Finally, we obtain that each iteration of OPERA takes $O(|\Delta\mathcal{R}| \cdot T_{R_r})$ time.

Thus, we obtain that the total running time of OPERA is upper bounded by $O(N_{\mathcal{R}} \cdot T_{R_r})$. This finishes the proof. □

From the analysis above, we obtain that the time complexity of OPERA is asymptotically the same as the state-of-the-art [20, 21].